

Dossier de evidencias — Milestone 2

Contrato COT-2026-TUV-001 · TUVET IA · Corte: 31 de julio de 2026

Este documento reúne la evidencia verificable del cumplimiento del Milestone 2. Cada afirmación va acompañada del identificador de commit y la fecha, de modo que cualquiera pueda comprobarla ejecutando ``git show <commit>`` sobre el repositorio.

Cómo leer este documento

Hay tres clases de evidencia y conviene no mezclarlas, porque tienen fuerza distinta:

- Evidencia documental (la más fuerte). Un commit con su fecha. No se puede alterar sin reescribir la historia del repositorio, y cualquier auditor la reproduce en segundos.
- Evidencia de estado. Una consulta a la base de datos de producción o una respuesta de un endpoint. Demuestra qué hay hoy, no desde cuándo.
- Evidencia de comportamiento. Una medición sobre un banco de casos, o un video del sistema funcionando. Demuestra que hace lo que se dice que hace.

Los apartados de abajo usan las tres. Donde una afirmación no tiene respaldo, se dice.

Resumen ejecutivo

El repositorio nace el 13 de julio de 2026 y el primer despliegue a producción (Railway para el backend, Vercel para el frontend) es del 16 de julio. Entre esas dos fechas ya estaban implementadas las dos garantías centrales del hito:

- Verificación de citas — ``verify_citations`` existe desde el primer commit (13-jul).
- Abstención — ``passes_threshold`` e ``insufficient_evidence`` existen desde el primer commit (13-jul).

Las dos semanas siguientes no fueron de implementación sino de CALIBRACIÓN: medir la primera versión, encontrar dónde fallaba y corregirla con datos. Esa distinción importa, porque explica por qué una función podía estar en producción y aun así no comportarse todavía como debía.

La abstención estaba implementada desde el día uno. Lo que no estaba era CALIBRADA: el umbral determinístico daba «hay evidencia» en 187 de 187 casos porque estaba saturado. Eso no es una función ausente, es una primera versión midiendo mal.

Línea de tiempo del proyecto

Cronología completa, de lo más antiguo a lo más reciente. Todos los identificadores son verificables con ``git show``.

Semana 1 — arquitectura y primer despliegue (13 al 16 de julio)

cca7b87 13-jul Entorno base del servicio + esquema RAG + scaffolding

6fd6de7 13-jul Migración 0002: índice vectorial HNSW (corpus_chunks, patient_embeddings)

fb922e6	14-jul	Núcleo determinístico: CITAS, cascada de recuperación, ingesta, gate de alergia
5c12135	14-jul	Tier 1 léxico+glosario, Tier 2 vector, contexto de paciente, tests cross-tenant
1433db8	14-jul	Generación B→A: armado, parseo y VERIFICACIÓN de la nota SOAP
2f3829d	15-jul	Endpoint /athos/phantom/suggest funcionando de punta a punta
f6660a1	16-jul	DESPLIEGUE: Railway (backend) + Vercel (frontend)

Al cierre de esta semana el sistema está EN PRODUCCIÓN con: recuperación en cascada, verificación de citas, gate de alergia severa, umbral de abstención y aislamiento por clínica cubierto por pruebas.

Semana 2 — corpus, módulos y primeras mediciones (21 al 27 de julio)

02f1d25	21-jul	El Fantasma no citaba por A→B pobre: banco golden de 4/11 a 9/11
a667277	22-jul	El Fantasma persiste alertas en clinical_notes (migración 0004)
ab385cc	23-jul	Agenda interna + sincronización con Google Calendar v1
1924924	23-jul	Vinculación de Google Calendar en un clic desde el login
5bc9e1b	23-jul	Captura de consulta con consentimiento (Ley 1581) + transcripción Deepgram
32ec1bf	24-jul	Invitaciones de equipo a la clínica
7e1d41c	27-jul	Reranking con Cohere + memoria semántica del paciente

El 26 de julio termina la ingesta del corpus completo en producción: 61.544 documentos, 519.999 fragmentos, el 100% con embedding.

Semana 3 — se mide, y la medición encuentra los problemas (28 al 29 de julio)

201972f	28-jul	El Tier 1 tardaba 15s de servidor; separando las ramas, 143 ms
58259da	28-jul	ABSTENCIÓN: juez semántico en bandas (none/limited/sufficient)
21cebd1	29-jul	Auditor de fidelidad de citas – apagado: sin calibrar descartaba el 58%
05d1bd0	29-jul	El rol del hablante se infiere del CONTENIDO, no de quién habló primero
47d02c9	29-jul	Documento de resultados del smoke testing del agente
8577be1	29-jul	24 de los 42 negativos no eran negativos: el instrumento estaba roto

Este es el punto de inflexión del proyecto. Al construir el instrumento de medición se descubre que varias funciones que existían no se comportaban como debían — y también que el propio instrumento estaba mal. Los dos hallazgos se corrigen.

Semana 4 — subsanación (30 y 31 de julio)

2303731	30-jul	Gemini integrado y cascada entre proveedores
a507043	30-jul	Routing POR CONSULTA (cláusula 1.5)
ca31838	30-jul	La cascada registra qué modelo respondió de verdad
aa7a72c	30-jul	Transcripción EN VIVO por WebSocket contra Deepgram
ac8fb8d	30-jul	El invitado sin cuenta ya puede entrar (defecto del enlace de correo)
cec3661	31-jul	Abstención: corroboración determinística – seguridad 82,4% a 92,6%
557232e	31-jul	Los 5 defectos de la conversación con el agente
97ef13c	31-jul	Banco de calidad del agente, corrido contra producción

Evidencia punto por punto

Punto 1 — Cascada y routing entre tres modelos

Implementado desde: 30 de julio de 2026

Situación previa

Hasta el 29 de julio el sistema operaba con un único proveedor (DeepSeek). No existía cascada ni routing entre modelos. Esto conviene reconocerlo de frente: la observación del cliente sobre este punto era correcta.

Qué se hizo

El 30 de julio se integró Gemini y se construyó la cascada entre proveedores (commit 2303731), se añadió el routing por consulta (a507043) y se amplió a los tres modelos con Claude (5cd027b). El 30 de julio se corrigió además el registro para que la traza guarde qué modelo respondió realmente (ca31838); antes anotaba un valor fijo, así que aunque hubiera habido fallback la traza no lo habría reflejado.

Estado hoy

Los tres proveedores están configurados en las variables de producción de Railway (GEMINI_API_KEY, ANTHROPIC_API_KEY, LLM_API_KEY) junto con las tres cadenas de cascada. Verificado con caída forzada contra los tres proveedores reales.

Prueba de caída forzada (30-jul)

Escenario	Quién responde	Latencia
Camino feliz	DeepSeek	1,4 s
Primario caído	Gemini	3,9 s
Primario y secundario caídos	Claude	3,7 s

Límite de la evidencia

La traza de producción registra 16 respuestas históricas, todas atendidas por DeepSeek. Cero por Gemini o Claude. Esto es coherente con el diseño —una cascada de respaldo sólo se activa cuando el primario falla, y no ha fallado— pero significa que no existe un registro de tráfico real que haya ejercitado el fallback. La evidencia disponible es la prueba de caída forzada, no el log de producción.

Cómo verificarlo

git show 2303731 · variables de Railway · docs/COMPARATIVA-MODELOS-2026-07-30.md

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 2 — Smoke testing del agente

Implementado desde: 29 de julio de 2026

Qué se hizo

El 29 de julio se construyó la suite de smoke testing del agente y su documento de resultados (commit 47d02c9). Son 22 casos que corren en integración continua en cada cambio.

Valor demostrado

La suite no es un trámite: al construirla encontró un defecto real que estaba en producción — la corrupción silenciosa de fechas, donde una fecha inválida como 2026-02-30 se agendaba el 2 de marzo sin avisar (commit ba2b3f9).

Estado hoy

22 casos en CI. Además, el ciclo agéntico completo quedó verificado en producción el 30 de julio: el asistente propuso una cita, apareció la tarjeta de aprobación, el veterinario aprobó y la acción quedó ejecutada con un identificador de cita real.

Cómo verificarlo

docs/AGENT-SMOKE-TESTING.md · `git show 47d02c9` · `git show ba2b3f9`

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 3 — Abstención — «cita o se calla»

Implementado desde: 13 de julio de 2026 (primera versión) · calibrada el 31 de julio

Implementación original

El mecanismo existe desde el PRIMER COMMIT del repositorio (cca7b87, 13 de julio): ``passes_threshold`` decide si hay evidencia suficiente y ``insufficient_evidence`` viaja en el contrato de la API hacia el frontend. Es decir, la función estaba implementada y desplegada desde el 16 de julio.

Qué falló en esa primera versión

Al medirla el 27 de julio sobre 187 casos se descubrió que el umbral daba «hay evidencia» en 187 de 187. La causa: el score estaba SATURADO por las constantes del Tier 1, y además el descriptor de especie («Dogs», presente en 43.000 fragmentos) contaba como evidencia temática. La función existía; su criterio no discriminaba.

Primera corrección

El 28 de julio (commit 58259da) se añadió un juez semántico que LEE los pasajes recuperados y dictamina en bandas: ninguna, limitada o suficiente. Se probaron antes tres señales gratuitas y las tres se descartaron con datos: el score determinístico (1.701 contra 1.700), el del reranker (0,532 contra 0,499) y el número de citas (6,0 contra 6,0). Ninguna discriminaba.

Segunda corrección

El 31 de julio se descubrió que el propio instrumento de medición estaba mal: el banco etiquetaba «¿el corpus contiene esto?» mientras la abstención decide «¿los pasajes recuperados cubren la consulta?». Sólo coinciden en el 74% de los casos. Se construyó una verdad mecánica basada en el árbol MeSH y se añadió una corroboración determinística (commit cec3661).

Evolución medida

Momento	Seguridad	Utilidad
Primera versión (umbral solo)	no discriminaba (187/187)	—
Con juez semántico	82,4 %	63,3 %
Con corroboración determinística	92,6 %	65,5 %
Mitad del banco no usada al calibrar	94,5 %	67,1 %

Límite de la evidencia

No existe el 100% en un juicio semántico: «¿esta literatura cubre este caso?» admite grados y dos veterinarios expertos discrepan en los bordes. Además el banco tiene 188 casos, así que un caso vale medio punto porcentual.

Cómo verificarlo

```
git show cca7b87 · git show 58259da · git show cec3661 · docs/ABSTENCION-MEDICION-2026-07-30.md
```

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 4 — Citas de fuentes correctas

Implementado desde: 13 de julio de 2026 (primera versión) · segunda capa el 29 de julio

Implementación original

La verificación de citas existe desde el PRIMER COMMIT (cca7b87, 13 de julio) y se consolida el 14 de julio en el núcleo determinístico (fb922e6). Está en producción desde el primer despliegue.

Por qué es una garantía estructural

El modelo NO puede escribir una fuente. Lo único que emite es un número entre corchetes. El título, el año, la revista y el enlace los reconstruye el código desde el fragmento recuperado de la base de datos. Un número que no corresponda a un documento realmente recuperado se descarta. Inventar una fuente no es improbable: no está representado en el camino de datos.

Segunda capa

El 29 de julio se añadió un auditor de fidelidad (commit 21cebd1) porque la procedencia sola no alcanzaba: medido, 18 de 24 respuestas citaban al menos un pasaje que no respaldaba la afirmación. Se entregó APAGADO por honestidad —sin calibrar descartaba el 58% de las referencias— y se encendió tras calibrarlo al 13-18%, sin dejar ninguna respuesta sin fuentes.

Límite de la evidencia

La segunda capa sólo QUITA referencias, nunca agrega ni sustituye. Su único error posible es ser demasiado estricta: jamás puede mostrar una cita incorrecta.

Cómo verificarlo

```
git show cca7b87 · git show fb922e6 · git show 21cebd1
```

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 5 — Latencia

Implementado desde: corregida el 28 de julio de 2026

El síntoma reportado

Se reportaron esperas de alrededor de cinco minutos.

La causa real

No era la infraestructura ni el plan contratado: era una consulta SQL. El Tier 1 del buscador combinaba dos ramas con un OR, lo que obligaba al motor a ordenar unos 19.000 resultados antes de aplicar el límite. Tardaba 15.397 milisegundos y se cancelaba por statement_timeout — lo que el usuario percibe como un cuelgue, no como lentitud.

La corrección

Separar las dos ramas (commit 201972f, 28 de julio). El mismo resultado en 143 milisegundos: 107 veces más rápido.

Medición actual

Qué	Medido
Consulta del Tier 1	143 ms (antes 15.397 ms)
Primer token del chat clínico	12,8 s
Respuesta completa	27,6 s
Carga de una página del frontend	0,4 s

Cómo verificarlo

```
git show 201972f
```

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 6 — Correo y módulo de comunicaciones

Implementado desde: 29 y 30 de julio de 2026

Qué se hizo

El motor de recaudo con los límites de la Ley 2300 (commit bd005fa, 29 de julio) y el envío real por SMTP con lectura de respuestas por IMAP (30 de julio). Incluye hilos de conversación, credenciales cifradas, pantalla de conexión, facturas por correo y recordatorios de cobranza.

Estado hoy

Cubierto por 8 pruebas automáticas. Cada clínica conecta su propia cuenta desde la pantalla de ajustes: que el módulo esté completo no significa que una clínica concreta ya lo haya conectado.

Cómo verificarlo

```
git show bd005fa · src/lib/cartera/channels.ts
```

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 7 — Google Calendar bidireccional

Implementado desde: 23 de julio de 2026

Implementación original

La agenda interna con sincronización hacia Google es del 23 de julio (commit ab385cc), junto con la vinculación en un clic desde el login (1924924). Las tablas de integración de calendario son del 23 de julio.

Qué faltaba

Las credenciales de Google (GOOGLE_CLIENT_ID y GOOGLE_CLIENT_SECRET) no estaban configuradas en producción, así que la sincronización no podía operar. Se configuraron el 31 de julio.

Estado hoy

Dos cuentas conectadas y 11.550 de 11.566 citas con identificador de evento de Google. La sincronización está operando sobre datos reales.

Límite de la evidencia

La traída automática desde Google hacia la plataforma requiere la verificación de la aplicación por parte de Google, que es un trámite de un tercero (~10 días). Mientras tanto funciona con un botón de sincronización manual.

Cómo verificarlo

git show ab385cc · consulta a appointments en producción

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 8 — Transcripción de consultas

Implementado desde: 23 de julio de 2026 · en vivo desde el 30 de julio

Implementación original

La captura de la consulta con consentimiento previo (Ley 1581) y la transcripción con Deepgram son del 23 de julio (commit 5bc9e1b), con corrección de la duración real del audio el mismo día.

Defecto 1 — roles invertidos

El código asumía que el hablante 0 era el veterinario «porque normalmente inicia la consulta». Es falso: el dueño suele abrir («Doctor, mi perro no come»), así que el diálogo entero salía invertido. Corregido el 29 de julio (commit 05d1bd0): el rol se infiere del CONTENIDO mediante marcadores lingüísticos, de forma determinística y auditable.

Defecto 2 — fecha errada

Tres pantallas formateaban en UTC porque los componentes de servidor corren en UTC. Ancladas a la zona horaria de Bogotá, con pruebas que fuerzan UTC para que no vuelva a ocurrir.

Defecto 3 — por lotes

El 30 de julio se implementó la transcripción EN VIVO por WebSocket contra Deepgram (commit aa7a72c). El veterinario ve el texto mientras habla.

Verificación contra Deepgram real

Métrica	Resultado
Exactitud	92,3 %
Exactitud ignorando formato numérico	96,1 %
Primer texto en pantalla	1,6 s
Fragmentos duplicados por reenvío	0

Límite de la evidencia

La prueba usó una consulta sintética de una sola voz, así que NO valida la separación de hablantes: Deepgram no tiene pista acústica para distinguir dos personas que suenan idénticas. La inferencia de rol sí tiene pruebas propias.

Cómo verificarlo

```
git show 5bc9e1b · git show 05d1bd0 · scripts/calidad/transcripcion_vivo_verificar.py
```

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 9 — Invitaciones de equipo

Implementado desde: 24 de julio de 2026 · defecto corregido el 30 de julio

Implementación original

Invitar colegas a la clínica por enlace y correo es del 24 de julio (commit 32ec1bf).

El defecto

Se reportó que el enlace no funcionaba. Se corrigieron cinco defectos en total. Los cuatro primeros: el destino no establecía sesión, el origen salía del dominio efímero del despliegue, la ruta de cierre de sesión respondía a GET (y el prefetch de un enlace cerraba la sesión sola) y el parámetro de destino podía quedar vacío.

El quinto, encontrado el 30 de julio

Quien recibía la invitación y NO tenía cuenta terminaba en la pantalla de inicio de sesión. La causa: un enlace de correo lo inicia el servidor, así que Supabase devuelve la sesión en el FRAGMENTO de la URL, y el fragmento nunca viaja al servidor. Corregido con una página cliente (commit ac8fb8d).

Por qué no se detectó antes

La invitación de prueba se aceptó con un correo que ya tenía cuenta: llegaba con sesión y el fallo no aparecía. Es una trampa de verificación que conviene conocer.

Cómo verificarlo

```
git show 32ec1bf · git show ac8fb8d · scripts/verificar_enlace_invitacion.py
```

[Espacio para adjuntar captura o enlace al video de este punto]

Punto 10 — Historial de conversaciones

Implementado desde: 13 de julio de 2026

Implementación original

La tabla `athos_messages` existe desde el PRIMER COMMIT (cca7b87, 13 de julio). Los mensajes se guardaron desde el principio.

El reclamo y la realidad

Se reportó que el historial «no persistía». Es incorrecto: sí persistía. Lo que faltaba era la PANTALLA que lo mostrara. Los datos estaban ahí todo el tiempo — hay 61 mensajes guardados desde el 16 de julio.

Estado hoy

La pantalla existe y precarga el hilo. El 31 de julio se añadió además la persistencia de las conversaciones del asistente agéntico, que era la única que faltaba (commit 557232e).

Cómo verificarlo

```
git show cca7b87 · consulta a athos_messages en producción
```

[Espacio para adjuntar captura o enlace al video de este punto]

El método — por qué se avanzó en ese orden

Las fechas de la sección anterior no son casualidad. Responden a cuatro decisiones técnicas que conviene explicar, porque son las que justifican por qué una función podía estar en producción el 16 de julio y aun así necesitar dos semanas más de trabajo.

1. Primero lo determinístico, después la inteligencia artificial

La arquitectura parte de una regla: gastar la menor inteligencia artificial posible. El buscador que recupera literatura es determinístico y no consume tokens; la IA sólo interviene en dos puntos — entender la consulta y redactar la respuesta citada.

Por eso la primera versión de la abstención (13 de julio) era un umbral numérico sobre el score del buscador: era lo barato, lo reproducible y lo que no dependía de ningún modelo. Sólo cuando se midió y se comprobó que no discriminaba se pagó el costo de un juez que LEE los pasajes.

Ese orden es deliberado, no un descuido. Introducir un modelo de lenguaje donde una regla determinística alcanza añade costo, latencia y una fuente de error que no se puede auditar. La secuencia correcta es intentar lo barato, medirlo, y escalar sólo si los datos lo exigen.

2. Las reglas duras viven en el código, no en el prompt

Está medido en este proyecto: pedirle al modelo por prompt que no dé una dosis sin peso del paciente falla en 2 de 23 casos, y con un prompt más resolutivo empeora a 9 de 23 —porque pedirle que decida lo empuja a decidir—. Por eso las garantías críticas están impuestas por código:

- El gate de alergia severa se calcula desde la tabla de alergias, no lo decide el modelo.
- El tapado de dosis cuando faltan datos lo hace un guard, no el prompt.
- La verificación de citas descarta referencias que no correspondan a un documento recuperado, sin preguntarle al modelo.
- El aviso de evidencia limitada lo emite el código cuando el juez dictamina esa banda.

Esto explica por qué varias correcciones de las últimas semanas fueron mover una regla desde el prompt hacia el código: no es refactorización cosmética, es la diferencia entre una garantía y una sugerencia.

3. Medir antes de construir — y arreglar el instrumento cuando está mal

Buena parte del trabajo del 28 al 31 de julio no fue escribir funciones nuevas sino construir con qué medir las existentes. Ese esfuerzo devolvió tres hallazgos que ninguna revisión de código habría encontrado:

- El umbral de abstención daba «hay evidencia» en 187 de 187 casos: la función existía pero su criterio estaba saturado.
- 18 de 24 respuestas citaban al menos un pasaje que no respaldaba la afirmación: la procedencia era correcta, la pertinencia no.

- 24 de los 42 casos negativos del banco de pruebas NO eran negativos: el instrumento de medición estaba roto, y toda cifra anterior medía otra cosa.

El tercero es el más incómodo y el más importante. Descubrir que el propio instrumento estaba mal obliga a descartar mediciones ya reportadas. Se hizo, y por eso las cifras de este documento son las que se sostienen.

4. Lo que se puede contar, se cuenta

Se probó usar un modelo de lenguaje como juez de calidad y se midió su fiabilidad: tiene un ruido de ± 7 sobre 40 puntos entre corridas idénticas, y un sesgo de posición del 78% hacia la primera opción mostrada. Es decir, el juez podía «demostrar» una mejora que no existía.

Desde entonces la regla es: si una propiedad se puede contar de forma determinística, se cuenta. La abstención se mide contra un hecho comprobable —si algún pasaje recuperado está indexado con el descriptor de la consulta en el árbol MeSH—, la transcripción contra una verdad de terreno conocida, y la proporcionalidad de las respuestas contando señales clínicas y caracteres. Ninguna cifra de este documento sale de la opinión de un modelo.

Guía para grabar los demos probatorios

Un video vale como evidencia sólo si es imposible discutir lo que muestra. Estas reglas existen para que nadie pueda decir «eso está editado» o «así no se mide».

Reglas que valen para todos los videos

- Una sola toma, sin cortes. Si algo sale mal, se repite el video entero. Un corte invalida la evidencia.
- Reloj visible en pantalla desde el primer segundo. Abrí time.is en una ventana lateral: muestra la hora oficial con segundos y sirve de sello temporal verificable.
- Mostrá la URL. Que se vea que es `tuvetia.vercel.app` y no un entorno local.
- Narrá lo que vas a hacer ANTES de hacerlo. «Voy a preguntar X y espero que pase Y.» Una predicción cumplida es más creíble que una explicación posterior.
- No edites nada. Ni recortes, ni acelerados, ni música. El archivo crudo.
- Nombrá el archivo con punto y fecha: ``P05-latencia-2026-08-01.mp4``.

Con qué grabar

En Windows, la Xbox Game Bar viene incluida: tecla Windows + G, y grabás con Windows + Alt + R. Si necesitás mejor calidad o mostrar el cronómetro con precisión, OBS Studio es gratuito y permite superponer un cronómetro sobre la pantalla.

Para el punto de latencia conviene además grabar con el panel de red del navegador abierto (F12, pestaña Red): ahí queda el tiempo real de cada petición, que es un dato que el propio navegador certifica.

Video por video: qué mostrar exactamente

P05 · Latencia (2 a 3 minutos)

- Mostrá time.is y la URL de producción.
- Abrí F12 en la pestaña Red y limpiá el registro.
- Escribí una consulta clínica real y decí en voz alta: «voy a enviar ahora».
- Envió y NO toques nada. Dejá que se vea aparecer el primer texto.
- Al aparecer el primer token, decilo en voz alta: «primer token».
- Cuando termine, mostrá en el panel de red el tiempo total de la petición.
- Cerrá mostrando el reloj otra vez.

Lo que prueba: que el primer token llega en torno a 12,8 s y la respuesta completa en 27,6 s. Los cinco minutos reportados no se reproducen.

[Espacio para pegar el video o su captura]

P03 · Abstención (3 a 4 minutos)

- Primero una consulta que Sí tenga literatura (por ejemplo, dermatitis atópica canina). Mostrá que responde citando fuentes.
- Después una consulta sin cobertura real en el corpus. Anunciá antes: «esto no debería tener evidencia suficiente».
- Mostrá el aviso de evidencia limitada o la abstención en pantalla.
- Si aparece la advertencia de evidencia limitada, leela en voz alta.

Lo que prueba: que el sistema distingue y lo declara. Es el contraste entre los dos casos lo que convence, no el segundo caso solo.

[Espacio para pegar el video o su captura]

P04 · Citas correctas (2 minutos)

- Hacé una consulta que devuelva fuentes.
- Hacé clic en una fuente y mostrá que abre el artículo real (PubMed o el enlace del documento).
- Volvé y abrí otra. Mostrá que el título y el año coinciden con lo que abre.

Lo que prueba: que las fuentes existen y son las recuperadas. El argumento fuerte es estructural —el modelo sólo emite un número, el resto lo reconstruye el código— pero verlo abrir el artículo real lo hace tangible.

[Espacio para pegar el video o su captura]

P08 · Transcripción en vivo (3 a 5 minutos)

- IMPORTANTE: grabá con DOS personas hablando, no una sola. Es lo único que demuestra la separación de hablantes.
- Mostrá la pantalla de consentimiento antes de grabar (Ley 1581).

- Inició la grabación y habló alternando: uno hace de dueño («Doctor, mi perro no come») y otro de veterinario («vamos a palpar el abdomen»).
- Mostró el texto apareciendo EN VIVO mientras hablan.
- Al detener, mostró que los roles quedaron bien asignados: Titular y Veterinario.
- Mostró la fecha de la consulta y comparala con el reloj en pantalla.

Lo que prueba: los tres defectos cerrados de una sola vez — en vivo, roles correctos y fecha correcta. Este es el video más valioso de todos, y el que cubre el hueco que la medición sintética no pudo cubrir.

[Espacio para pegar el video o su captura]

P01 · Cascada de tres modelos (3 minutos)

- Este NO se puede demostrar desde la interfaz: el fallback sólo se activa si el proveedor principal falla.
- Grabó en su lugar la ejecución de la prueba de caída forzada en la terminal, mostrando que responde DeepSeek, luego Gemini y luego Claude.
- Mostró también las variables de entorno de Railway con las tres claves configuradas (sin revelar los valores: basta con que se vean los nombres).

Lo que prueba: que la cascada existe y funciona. Sé explícito en el video sobre que es una prueba de caída forzada y no tráfico real: es lo que hace creíble todo lo demás.

[Espacio para pegar el video o su captura]

P07 · Google Calendar (2 a 3 minutos)

- Mostró el calendario de la plataforma con las citas.
- Abrí Google Calendar en otra pestaña y mostró las mismas citas.
- Creó una cita en la plataforma y mostró cómo aparece en Google.

Lo que prueba: la sincronización operando. Hay 11.550 citas sincronizadas, así que el volumen habla solo.

[Espacio para pegar el video o su captura]

P09 · Invitaciones (2 minutos)

- Usá un correo que NO tenga cuenta en la plataforma — es el caso que estaba roto.
- Enviá la invitación desde ajustes de equipo.
- Abrí el correo, hacé clic en el enlace.
- Mostrá que aterriza en la invitación y que al aceptar entra a la clínica.

Lo que prueba: el quinto defecto corregido. Con un correo que ya tenga cuenta NO se demuestra nada: ése es el camino que siempre funcionó.

[Espacio para pegar el video o su captura]

P10 · Historial y memoria (2 minutos)

- Escribí una pregunta al asistente y esperá la respuesta.
- RECARGÁ la página por completo (F5).
- Mostrá que la conversación anterior sigue ahí.
- Preguntá: «¿cuál fue la última pregunta que te hice?» y mostrá que responde bien.

Lo que prueba: que el historial persiste entre sesiones. La recarga es la parte importante: sin ella no se demuestra persistencia.

[Espacio para pegar el video o su captura]

Qué NO hacer

- No grabes en un entorno local. Si la URL no es la de producción, no prueba nada.
- No uses datos de pacientes reales de una clínica cliente sin autorización. Creá pacientes de demostración.
- No repitas la toma hasta que «salga bien» y muestres sólo esa. Si el sistema falla en un intento, eso también es información — y si se descubre después, cuesta mucho más caro.
- No afirmes en el video nada que el video no muestre. Si querés dar contexto, decilo como contexto, no como demostración.

Anexo — comandos de verificación

Cualquiera con acceso al repositorio puede reproducir esta evidencia.

Verificar un commit concreto

```
git show cca7b87          # primer commit: citas + abstención + historial
git log --format='%h %ad %s' --date=short --reverse | head -20
```

Correr la suite completa

```
python scripts/auditoria.py          # backend + front + tipos + lint + build
```

Medir la abstención contra producción

```
cd athos-service
python scripts/calidad/abstencion_verdad.py --n 0
```

Medir la transcripción en vivo contra Deepgram

```
python scripts/calidad/transcripcion_vivo_verificar.py audio.wav referencia.txt
```

Correr el banco de calidad del agente

```
RUN_BANCO=1 ANTHROPIC_API_KEY=... \
npx vitest run --config vitest.e2e.config.mts e2e/banco-agente.e2e.ts
```

Comprobar la configuración de producción

```
curl -H "Authorization: Bearer $CRON_SECRET" \
https://tuvetia.vercel.app/api/health
```

Documentos de referencia en el repositorio

- docs/SOPORTES-MILESTONE2-2026-07-30.md — respuesta punto por punto a la guía de soportes
- docs/VERIFICACION-10-PUNTOS-2026-07-30.md — estado de los 10 puntos priorizados
- docs/ABSTENCION-MEDICION-2026-07-30.md — cómo se mide la abstención y por qué
- docs/BANCO-AGENTE-RESULTADO.md — banco de calidad del agente con las respuestas íntegras
- docs/COMPARATIVA-MODELOS-2026-07-30.md — prueba de caída forzada entre los tres proveedores
- docs/AGENT-SMOKE-TESTING.md — resultados del smoke testing
- docs/CONFIGURACION-PRODUCCION.md — qué variable vive dónde
- INVENTARIO-COMPONENTES.md — inventario formal de componentes